Technische Hochschule Augsburg

Fakultät für Informatik

Thesis

zur Erlangung des Grades
Bachelor of Engineering in Systems Engineering

Proximitätssuche von EPLAN-Makros auf Basis des LambdaRank-Algorithmus

erstellt von Andreas Alstetter

Andreas Alstetter

Pfarrer-Kempter Str. 4
89358 Kammeltal/Behlingen
T +4915201472197
andreas.alstetter@tha.de

Matrikelnummer:
2147455

Technische Hochschule Augsburg

An der Hochschule 1
D-86161 Augsburg
T +49 821 5586-0
F +49 821 5586-3222
www.tha.de
info@tha.de

Erstprüfer	Prof. Dr. Thomas Kirchmeier
Zweitprüfer	Prof. Dr. Jürgen Grinninger
Themenvergabe am	16. Juni 2026
Eingereicht am	November 4, 2026
Geheimhaltungsvereinbarung	Nein

Inhaltsverzeichnis

1 Motivation 1
1.1 Herleitung . . . 1
1.2 Vision . . . 1
1.3 Herausforderung . . . 1

2 Stand der Technik 2
2.1 Large Language Models . . . 2
2.2 Fine-Tuning . . . 2
2.3 Sentence Transformers . . . 2
2.4 Embedding-Modelle . . . 2
2.5 Triplet Loss . . . 2
2.6 Learning-to-Rank . . . 2
2.6.1 Grundprinzip von Learning-to-Rank . . . 2
2.6.2 LambdaRank . . . 2
2.7 Recall@k . . . 2
2.8 Mean Reciprocal Rank . . . 3
2.9 Normalized Discounted Cumulative Gain . . . 4
2.10 Hierarchische Bewertungsmetriken . . . 4

3 Handlungsbedarf und Forschungsfrage 6
3.1 Handlungsbedarf . . . 6
3.2 Forschungsfrage . . . 6

4 Vorgehensweise 7
4.1 Methodisches Vorgehen . . . 7
4.2 Iterativer Entwicklungsprozess . . . 7

5 Architekturübersicht 10
5.1 Empfehlungssystem auf Basis eines Large Language Models mit Fine-Tuning . 10
5.2 Empfehlungssystem auf Basis eines Sentence Transformers . . . 10
5.3 Empfehlungssystem auf Basis eines Embedding-Modells mit Triplet Loss . . . 10
5.4 Empfehlungssystem auf Basis eines Learning-to-Rank-Modells . . . 10

6 Datenanalyse und Trainigsvorbereitung 11
6.1 Analyse der Ausgangsdaten . . . 11

6.2 Aufbereitung der Trainingsdaten . . . 11
6.3 Datensplit . . . 11
6.4 Trainingsbasis . . . 11

7 Auswahl des Modell-Ansatzes 12
7.1 Gegenüberstellung der Modell-Ansätze . . . 12
7.1.1 Large Language Model mit Fine-Tuning . . . 12
7.1.2 Sentence-Transformer . . . 12
7.1.3 Embedding-Modell mit Triplet Loss . . . 12
7.1.4 Learning-to-Rank-Modell . . . 12
7.2 Modellauswahl . . . 12

8 Training des Learning-to-Rank-Modells 13
8.1 Aufbau der Trainingsmatrix . . . 13
8.2 Parameter Suche des Learning-to-Rank-Modells . . . 13
8.3 Hyperparameteroptimierung . . . 15

9 Modellbewertung 16
9.1 Versuchsserie A01–A09 . . . 16
9.1.1 Untersuchte Konfigurationsparameter . . . 16
9.1.2 Referenzkonfiguration A01 . . . 17
9.1.3 Ergebnisse . . . 18
9.1.4 Same-Family-Gewicht . . . 18
9.1.5 Hierarchische Labeldefinition . . . 19
9.1.6 Profilbreite . . . 19
9.1.7 Regularisierung . . . 19
9.1.8 Top-Konfiguration . . . 20

10 Diskussion der Ergebnisse 21
10.1 Interpretation der Ergebnisse . . . 21
10.2 Ursachenanalyse der Rankingfehler . . . 21

11 Grenzen und Ausblick 23
11.1 Grenzen des aktuellen Systems . . . 23
11.1.1 Geringer Support von Ziel-Makros . . . 23
11.1.2 Variantenauflösung innerhalb einer Makrofamilie . . . 24
11.1.3 Support-Bias häufiger Ziel-Makros . . . 24
11.2 Ausblick . . . 24

12 Zusammenfassung und Fazit 26

Literaturverzeichnis 30

1 Motivation

1.1 Herleitung

Prozesse werden in der Industrie zunehmend komplexer und dynamischer. Komplexität übersteigt die menschliche Wahrnehmungs- und Verarbeitungskapazität. Dadruch sinkt die effizienz und effektivität von Entscheidungs und Entwicklungsprozessen. Durch die Verwendung von KI + Maschniellem Lernen, kann die Komplexität der Prozesse durch die Nutzung und Analyse großer Datenmengen gesteuert werden.

1.2 Vision

1.3 Herausforderung

2 Stand der Technik

2.1 Large Language Models

2.2 Fine-Tuning

2.3 Sentence Transformers

2.4 Embedding-Modelle

2.5 Triplet Loss

2.6 Learning-to-Rank

2.6.1 Grundprinzip von Learning-to-Rank

Learning-to-Rank (LTR) bezeichnet eine Klasse von maschinellen Lernverfahren, die das Ziel besitzen, die Reihenfolge in einer Ergebnisliste zu optimieren (Liu 2009). Im Gegensatz zu klassischen Klassifikations- oder Regressionsansätzen bewertet Learning-to-Rank keine einzelnen Objekte isoliert, sondern die relative Relevanz dieser Objekte in einer Kandidatenmenge. Die Aufgabe eines Rankingmodells besteht darin, dass für eine Anfrage die relevanten Dokumente anhand ihrer Merkmale bewertet werden und anschließend in eine Rangliste überführt werden.

2.6.2 LambdaRank

2.7 Recall@k

Recall@k gehört zu den grundlegenden Bewertungsmetriken des Information Retrievals und beschreibt den Anteil der Anfragen, bei denen mindestens ein relevantes Ergebnis innerhalb der

ersten k Rangpositionen gefunden wird (Manning u. a. 2008). Diese Metrik berechnet sich wie folgt:

$$Recall@k = \frac{\text{\#Anfragen mit relevantem Ergebnis in Top-}k}{\text{\#Anfragen}} \tag{2.1}$$

mit

k : Anzahl der betrachteten Rangpositionen
Anfragen mit relevantem Ergebnis in Top-k : Anzahl erfolgreicher Anfragen
Anfragen : Gesamtzahl aller Anfragen

Der Wertebereich liegt zwischen 0 und 1. Ein Wert von 1 bedeutet, dass sich für jede Anfrage mindestens ein relevantes Ergebnis innerhalb der Top-k Positionen der Rangliste befindet. In Ranking- und Empfehlungssystemen wird Recall@k häufig verwendet, um die praktische Verwendung eines Systems beurteilen zu können. Der Anwender betrachtet dabei in der Regel nur die ersten Ergebnisse einer Rangliste (Manning u. a. 2008).

2.8 Mean Reciprocal Rank

Der Mean Reciprocal Rank (MRR) zählt neben Recall@k zu etablierten Bewertungsmetriken des Information Retrievals und wird sowohl in klassischen Suchsystemen als auch in modernen Rankingverfahren eingesetzt (Voorhees und Tice 1999; Zhu u. a. 2024). Mit dieser Kennzahl wird die Position des ersten relevanten Ergebnisses innerhalb einer Rangliste bewertet.

Die Berechnung ergibt sich wie folgt:

$$MRR = \frac{1}{N} \sum_{i=1}^{N} \frac{1}{rank_i} \tag{2.2}$$

mit

N Anzahl der ausgewerteten Anfragen
$rank_i$ Rangposition des ersten relevanten Ergebnisses für Anfrage i

Der Wertebereich liegt zwischen 0 und 1. Ein hoher Wert zeigt an, dass ein relevantes Ergebnis bereits an einer der vorderen Positionen einer Rangliste erscheint. Da frühe Treffer stärker gewichtet werden als spätere Treffer, eignet sich die Kennzahl besonders zur Bewertung der schnellen Auffindbarkeit relevanter Ergebnisse.

2.9 Normalized Discounted Cumulative Gain

Der Discounted Cumulative Gain (DCG) wurde von Järvelin und Kekäläinen zur Bewertung verschiedener Relevanzstufen von Suchergebnissen eingeführt und zählt zu den wichtigsten Bewertungsmetriken für Ranking- und Suchsysteme (Järvelin und Kekäläinen 2002). Die Kennzahl berücksichtigt sowohl die Relevanz eines Ergebnisses als auch die Position innerhalb der Rangliste. Relevante Ergebnisse auf frühen Rangpositionen leisten dadurch einen höheren Beitrag zur Gesamtbewertung.

Die Berechnung des DCG erfolgt über

$$DCG@k = \sum_{i=1}^{k} \frac{2^{rel_i} - 1}{\log_2(i+1)} \tag{2.3}$$

mit

k Anzahl der betrachteten Rangpositionen
i Rangposition innerhalb der Rangliste
rel_i Relevanzwert des Ergebnisses auf Rangposition i

Um eine bessere Vergleichbarkeit zu schaffen, findet eine Normierung auf die ideale Rangliste statt. Hieraus ergibt sich der Normalized Discounted Cumulative Gain (nDCG):

$$nDCG@k = \frac{DCG@k}{IDCG@k} \tag{2.4}$$

mit

$DCG@k$ DCG-Wert der betrachteten Rangliste
$IDCG@k$ *Ideal Discounted Cumulative Gain*, also der höchstmögliche DCG-Wert, der bei einer idealen Sortierung der Kandidaten nach absteigender Relevanz erreicht würde

Der Wertebereich liegt zwischen 0 und 1. Ein Wert von 1 entspricht der optimal möglichen Rangfolge.

2.10 Hierarchische Bewertungsmetriken

Nicht jede fachliche Fehlentscheidung hat in technischen Empfehlungssystemen dieselbe Bedeutung. Ein Ergebnis kann beispielsweise einer technisch verwandten Produkt- oder Makrofamilie

angehören, aber trotz fehlender exakter Übereinstimmung für den Anwender weiterhin von Interesse sein. Zur Berücksichtigung solcher Beziehungen werden hierarchische Bewertungsmetriken eingesetzt.

Diese basieren typischerweise auf dem Discounted Cumulative Gain (Järvelin und Kekäläinen 2002) und verwenden unterschiedliche Relevanzen in abgestufter Reihenfolge.

Eine mögliche Relevanzfunktion lautet:

$$rel = \begin{cases} 3, & \text{exakte Übereinstimmung} \\ 2, & \text{gleiche Familie} \\ 0, & \text{keine fachliche Beziehung} \end{cases} \tag{2.5}$$

Dadurch wird eine Rangliste bevorzugt, welche fachlich ähnliche Kandidaten auf höheren Positionen platziert. Hierarchische Bewertungsmetriken eignen sich insbesondere für industrielle Empfehlungssysteme, bei denen zwischen exakter Übereinstimmung und fachlicher Nähe unterschieden werden muss.

3 Handlungsbedarf und Forschungsfrage

3.1 Handlungsbedarf

3.2 Forschungsfrage

4 Vorgehensweise

4.1 Methodisches Vorgehen

Das methodische Vorgehen orientiert sich an der *Design Science Research Methodology* (DSRM) nach Peffers et al. (2007). Die DSRM beschreibt einen strukturierten Forschungsprozess zur Entwicklung und Evaluation technischer Artefakte und eignet sich insbesondere für die Entwicklung von IT-Systemen. Das Ziel der Methodik besteht darin, eine konkrete Problemstellung systematisch zu analysieren, ein geeignetes Artefakt zu entwickeln und den Nutzen durch eine Evaluation nachzuweisen.

Der von Peffers et al. beschriebene Forschungsprozess umfasst die folgenden Schritte:

1. Problemidentifikation und Motivation
2. Definition der Lösungsziele
3. Design und Entwicklung des Artefakts
4. Demonstration der entwickelten Lösung
5. Evaluation des Artefakts
6. Kommunikation der Ergebnisse

In dieser Arbeit bildet die DSRM den übergeordneten Rahmen. Die iterative Entwicklung des Ranking-Systems erfolgt innerhalb der Design-, Entwicklungs- und Evaluationsphase durch wiederkehrende Entwicklungszyklen.

4.2 Iterativer Entwicklungsprozess

Zur Entwicklung des Artefakts wird die Design-Science-Methodik durch die Prinzipien des *Action Design Research* (ADR) nach Sein et al. (2011) verwendet. ADR verbindet dabei die Entwicklung eines Artefakts mit der fortlaufenden Anwendung, Evaluation und Verbesserung. Während die DSRM den strukturellen Prozess beschreibt, liefert ADR den Entwicklungszyklus zur kontinuierlichen Verbesserung des Systems.

Der iterative Entwicklungsprozess besteht dabei aus den folgenden Schritten:

1. Analyse des aktuellen Zustands und Identifikation von Verbesserungspotentialen

2. Recherche wissenschaftlicher und technischer Lösungen
3. Konzeption neuer Komponenten, Merkmale oder Optimierungen
4. Implementierung und Anpassung der entwickelten Ansätze
5. Durchführung von Tests und Validierungen
6. Evaluation anhand definierter Qualitätsmetriken
7. Interpretation der Ergebnisse
8. Ableitung weiterer Verbesserungen

Die Ergebnisse einer Iteration bilden jeweils die Grundlage für die nachfolgenden Iterationen. Auf diese Weise entsteht ein kontinuierlicher Verbesserungsprozess, in dem Forschung, Entwicklung, Evaluation und Optimierung miteinander verzahnt sind. Die Kombination aus DSRM und ADR ermöglicht somit auf der einen Seite einen strukturierten Forschungsprozess und auf der anderen Seite eine iterative Verbesserung des entwickelten Ranking-Systems. Abbildung 4.1 veranschaulicht diese Verzahnung der beiden Methodiken.

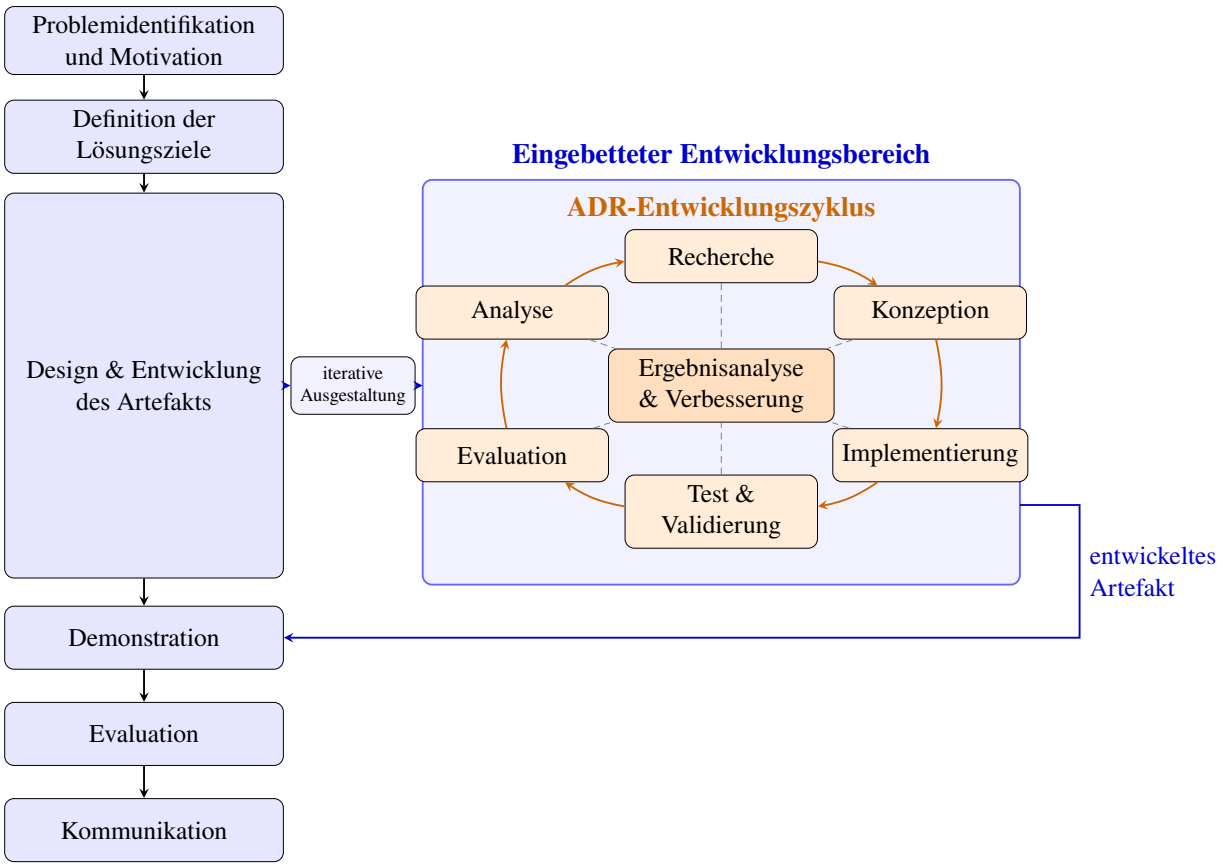


Abbildung 4.1: Verzahnung von DSRM und ADR. Die DSRM bildet den übergeordneten Forschungsrahmen, während ADR als iterativer Entwicklungszyklus innerhalb der Phase „Design & Entwicklung des Artefakts“ eingesetzt wird.

Die theoretischen Grundlagen der verwendeten Bewertungsmetriken wurden bereits in Kapitel 2

vorgestellt. Die Anwendung dieser Metriken zur Bewertung des entwickelten Systems erfolgt in Kapitel **??**. Eine Darstellung der finalen Systemarchitektur und der vollständigen Pipeline befindet sich ebenfalls in diesem Kapitel.

5 Architekturübersicht

5.1 Empfehlungssystem auf Basis eines Large Language Models mit Fine-Tuning

5.2 Empfehlungssystem auf Basis eines Sentence Transformers

5.3 Empfehlungssystem auf Basis eines Embedding-Modells mit Triplet Loss

5.4 Empfehlungssystem auf Basis eines Learning-to-Rank-Modells

6 Datenanalyse und Trainigsvorbereitung

6.1 Analyse der Ausgangsdaten

Eplan Dateien, .ema, Artikel, Makros, xml-format + Aufbau ...

6.2 Aufbereitung der Trainingsdaten

6.3 Datensplit

6.4 Trainingsbasis

7 Auswahl des Modell-Ansatzes

7.1 Gegenüberstellung der Modell-Ansätze

7.1.1 Large Language Model mit Fine-Tuning

7.1.2 Sentence-Transformer

7.1.3 Embedding-Modell mit Triplet Loss

7.1.4 Learning-to-Rank-Modell

7.2 Modellauswahl

8 Training des Learning-to-Rank-Modells

8.1 Aufbau der Trainingsmatrix

Nach der Erstellung der Kandidatenprofile werden für jede Anfrage die verfügbaren Makrokandidaten bewertet. Für jede Anfrage werden Anfrage-Kandidat-Paare erzeugt und anhand der Pairwise-Merkmale aus Kapitel **??** beschrieben. Die Referenzkonfiguration A08 verwendet insgesamt 375 Kandidatenprofile und 2.381 Merkmale. Für jede Anfrage werden alle 375 Kandidatenprofile vollständig bewertet.

Tabelle 8.1 fasst die Größenordnung der erzeugten Rankingmatrizen zusammen.

Tabelle 8.1: Dimensionen der erzeugten Rankingmatrizen

Split	Anfragen	Kandidaten je Anfrage	Matrixzeilen	Merkmale
Training	984	375	369.000	2.381
Validation	322	375	120.750	2.381
Test	253	375	94.875	2.381

Der erzeugte Artikel-Kandidat-Merkmalsraum ist sehr groß und viele Merkmalswerte besitzen den Wert null. Aus diesem Grund werden Sparse-Matrizen verwendet. Bei einer Sparse-Matrix werden nicht belegte Werte nicht vollständig gespeichert, wodurch der Speicherbedarf reduziert wird. Die numerischen Werte werden im 32-Bit-Gleitkommaformat `float32` verarbeitet.

8.2 Parameter Suche des Learning-to-Rank-Modells

Das zentrale Element dieser Architektur basiert auf LightGBM. LightGBM ist eine Bibliothek für Gradient Boosting mithilfe von Entscheidungsbäumen. Dabei werden sequentiell Entscheidungsbäume erzeugt, welche die Fehler des bisherigen Modells schrittweise korrigieren. Für Rankingaufgaben ist der `LGBMRanker` besonders geeignet. Dieser ist auf Rankinggruppen sowie relative Ergebnisreihenfolgen spezialisiert und verwendet in diesem Projekt das LambdaRank-Verfahren (Microsoft Corporation 2026). Dieser basiert auf dem in Kapitel 2 beschriebenen LambdaRank-/LambdaMART-Prinzip.

Das Modell wird mit den Parametern

$$\text{objective} = \text{lambdarank} \qquad \text{und} \qquad \text{metric} = \text{ndcg} \tag{8.1}$$

trainiert (Pipeline step 931 2026).

Der Parameter `objective` legt das Optimierungsziel des Modells fest. Mit `lambdarank` wird die Optimierung der Kandidatenreihenfolge durch LambdaRank aktiviert. Der Parameter `metric` bestimmt die verwendete Rankingmetrik der Bewertung; `ndcg` steht für Normalized Discounted Cumulative Gain und wird im Kapitel 2 näher erläutert.

Die für die spätere Referenzkonfiguration A08 ausgewählten Modellparameter lauten:

$$n_estimators = 900 \tag{8.2}$$
$$learning_rate = 0.025 \tag{8.3}$$
$$num_leaves = 15 \tag{8.4}$$
$$min_child_samples = 50 \tag{8.5}$$
$$lambdarank_truncation_level = 5 \tag{8.6}$$

Zusätzlich wird eine Kandidatenprofilbreite von 16 Artikeln pro Makro verwendet.

Wie in Kapitel **??** beschrieben, werden Kandidaten einer Anfrage als gemeinsame Rankinggruppe betrachtet. Entsprechend entsteht für jede Anfrage eine Gruppe:

$$G_q = \{(q, c_1), (q, c_2), \ldots, (q, c_n)\} \tag{8.7}$$

mit der Anfrage q und den zugehörigen Makrokandidaten c_i.

Dadurch erlernt das Modell nicht die allgemeine Zugehörigkeit eines einzelnen Kandidaten zu einer Klasse. Stattdessen wird die optimale Kandidatenreihenfolge innerhalb derselben Anfrage gelernt. Ziel dabei ist es, dass der relevanteste Kandidat möglichst weit oben in der Rangliste erscheint.

Die Eingaben für das Modell sind die im Kapitel **??** beschriebenen Pairwise-Merkmale. Das Training erfolgt dabei ausschließlich auf den Trainingsdaten, während die Optimierung der Hyperparameter über die Validierungsdaten erfolgt (Pipeline step 931 2026).

8.3 Hyperparameteroptimierung

Für die Hyperparameteroptimierung wird eine Grid-Search mit insgesamt 15 definierten Parameterkonfigurationen verwendet. Ein Grid-Search ist die systematische Erprobung von zuvor festgelegten Kombinationen an Modellparametern. Jede Kombination wird dabei trainiert und anhand des Validierungsbestands bewertet; anschließend wird anhand definierter Rankingmetriken die beste Konfiguration ausgewählt. Variiert werden dabei folgende Parameter:

- `n_estimators`,
- `learning_rate`,
- `num_leaves`,
- `min_child_samples`,
- `lambdarank_truncation_level`.

Die Tabelle 8.2 zeigt die untersuchten Hyperparameterkombinationen.

Tabelle 8.2: Variierte Hyperparameter der Grid-Search (Pipeline step 931 2026)

Parameter	**Wertebereich**
`n_estimators`	300 bis 900
`learning_rate`	0.025 bis 0.05
`num_leaves`	15 bis 127
`min_child_samples`	5 bis 80
`lambdarank_truncation_level`	5 bis 20

Dabei sind die Hyperparameter wie folgt zu verstehen:

- **`n_estimators`:** Anzahl der aufgebauten Entscheidungsbäume im Modell. Mehr Bäume = komplexere Zusammenhänge, aber mehr Rechenaufwand.
- **`learning_rate`:** Lernrate pro Trainingsschritt. Kleinere Werte führen zu vorsichtigerem, aber stabilerem Lernen.
- **`num_leaves`:** Maximale Anzahl der Blätter pro Baum. Mehr Blätter führen zu komplexeren Bäumen und erhöhen dadurch das Risiko von Overfitting.
- **`min_child_samples`:** Minimale Anzahl an Trainingsbeispielen in einem Blatt. Kleinere Werte = mehr Detailtreue pro Blatt, größere Werte = bessere Regularisierung (verhindert Overfitting).
- **`lambdarank_truncation_level`:** Begrenzung des Ranglistenbereichs, der durch LambdaRank optimiert werden soll (Top 10, Top 20).

9 Modellbewertung

9.1 Versuchsserie A01–A09

Um verschiedene Modellkonfigurationen zu vergleichen, wurde die aktuelle Versuchsserie A01 bis A09 durchgeführt. Dabei wurden unterschiedliche Strategien, Profilbreiten und Regularisierungsparameter getestet. Ziel davon ist es, die Einflüsse einzelner Parameter auf die Modellleistung zu analysieren, (Alstetter 2026).

9.1.1 Untersuchte Konfigurationsparameter

Die verschiedenen Varianten der Konfiguration haben unterschiedliche Auswirkungen auf die Trainingslabels, die Kandidatenprofile sowie die Modellregularisierung. Die untersuchten Konfigurationsparameter werden in Tabelle 9.1 erläutert.

Tabelle 9.1: Untersuchte Konfigurationsparameter

Parameter	**Bedeutung**
Same-Family-Gewicht	Negative Kandidaten aus derselben Makrofamilie werden innerhalb der Verlustfunktion unterschiedlich gewichtet. Dadurch kann die Variantenauflösung stärker betont werden.
Profilbreite	Begrenzt die Anzahl der gespeicherten Profilwerte innerhalb eines Kandidatenprofils. Kleinere Profilbreiten erzeugen eine kompaktere Kandidatenrepräsentation.
L1-Regularisierung (α)	Bestraft große Modellgewichte und reduziert die Modellkomplexität. Dadurch werden irrelevante Merkmale stärker unterdrückt.
L2-Regularisierung (λ)	Stabilisiert die Gewichtung der einzelnen Merkmale und reduziert die Gefahr einer Überanpassung an einzelne Trainingsbeispiele.
Min. Split Gain	Definiert den minimal notwendigen Informationsgewinn, um eine Baumaufteilung im LightGBM-Modell zu ermöglichen. Höhere Werte führen dadurch zu konservativeren Entscheidungsbäumen.

Bei der L1-Regularisierung wird die Summe der Absolutwerte der Modellgewichte bestraft:

$$\lambda_1 \sum_j |w_j|. \tag{9.1}$$

Dadurch werden einzelne Gewichte auf null gesetzt. Die L2-Regularisierung bestraft dagegen die Summe der quadrierten Gewichte:

$$\lambda_2 \sum_j w_j^2. \tag{9.2}$$

Dadurch werden besonders große Gewichte begrenzt. Bei LightGBM beziehen sich diese Regularisierungsstrafen hauptsächlich auf die Ausgabewerte der Baumblätter.

A02 bis A07 wurden so gewählt, dass immer nur ein Parameter gezielt verändert wurde. Dadurch lassen sich die Auswirkungen der einzelnen Konfigurationsparameter isoliert untersuchen. Bei A08 und A09 werden mehrere Parameter (L1 und L2) gleichzeitig verändert, da diese bei LightGBM konzeptionell denselben Hebel darstellen (Bestrafung der Blattgewichte). Um eine stärkere Regularisierung zu erzielen, ist es in der Praxis sinnvoll, beide gemeinsam hochzudrehen, nicht nur einen isoliert. Bei A09 kommt zusätzlich ein positiver Wert für `min_split_gain` zum Einsatz, um dadurch einen noch größeren Einfluss auf eine stärkere Regularisierung zu erzielen, (Alstetter 2026).

9.1.2 Referenzkonfiguration A01

A01 dient als Referenzkonfiguration und bildet die Basis für die Vergleichsexperimente. Sie enthält die Standardwerte für alle untersuchten Konfigurationsparameter. Alle weiteren Konfigurationen A02 bis A09 werden gegenüber dieser bewertet.

Die Referenzparameter sind in Tabelle 9.2 zusammengefasst.

Tabelle 9.2: Referenzkonfiguration A01

Parameter	**Wert**
Exaktes Targettraining	Ja
Same-Family-Gewicht	5,0
Profilbreite	16
L1-Regularisierung (α)	0,05
L2-Regularisierung (λ)	0,2
Min. Split Gain	0,0

Ausgehend von dieser Basis werden die Varianten der Konfigurationen A02 bis A09 verändert. Tabelle 9.3 zeigt die gegenüber A01 vorgenommenen Veränderungen.

Tabelle 9.3: Abweichungen der Konfigurationen gegenüber der Referenz A01

Konfiguration	Änderung gegenüber A01
A02	Same-Family-Gewicht von 5,0 auf 2,0 reduziert
A03	Same-Family-Gewicht von 5,0 auf 8,0 erhöht
A04	Same-Family-Gewicht von 5,0 auf 12,0 erhöht
A05	Hierarchische Relevanzlabels statt ausschließlich exakter Ziel-Makros
A06	Profilbreite von 16 auf 12 reduziert
A07	Profilbreite von 16 auf 8 reduziert
A08	L1-Regularisierung auf 0,1 und L2-Regularisierung auf 0,5 erhöht
A09	L1-Regularisierung auf 0,5, L2-Regularisierung auf 1,0 und `min_split_gain` auf 0,01 erhöht

Dadurch lässt sich der Einfluss der jeweiligen Parameteränderung der entsprechenden Konfiguration zuordnen, (Alstetter 2026).

9.1.3 Ergebnisse

Tabelle 9.4 zeigt die getesteten Modellkonfigurationen und deren Leistung im Vergleich.

Tabelle 9.4: Ergebnisse der A01–A09-Konfigurationen auf dem Testdatensatz

Konfiguration	Recall@1	Recall@3	Recall@10	MRR
A01 Referenz	0,3241	0,4822	0,6798	0,4418
A02 Gewicht 2	0,3162	0,4862	0,6680	0,4399
A03 Gewicht 8	0,3241	0,4743	0,6838	0,4397
A04 Gewicht 12	0,2925	0,4783	0,6759	0,4195
A05 Hierarchisch	0,3162	0,4783	0,6719	0,4328
A06 Profilbreite 12	0,3162	0,4862	0,6877	0,4374
A07 Profilbreite 8	0,3241	0,4862	0,6719	0,4439
A08 Regularisierung mittel	**0,3478**	0,4783	0,6798	**0,4534**
A09 Regularisierung stark	0,3399	0,4783	**0,6917**	0,4505

9.1.4 Same-Family-Gewicht

Mit den Konfigurationen A02 bis A04 wird der Einfluss des Same-Family-Gewichts untersucht. Die Ergebnisse zeigen, dass eine Verringerung des Gewichts auf 2 den Recall@1 auf 31,62 % senkt, während eine Erhöhung auf 8 keine Auswirkung zeigt. Die Gewichtung auf 12 führt zu einer deutlichen Verschlechterung des Recall@1 auf 29,25 %.

Daraus lässt sich schließen, dass ein zu hohes Same-Family-Gewicht die Lernstabilität negativ beeinflusst.

9.1.5 Hierarchische Labeldefinition

A05 untersucht die hierarchische Relevanzdefinition. Makros (EPLAN GmbH & Co. KG 2025) derselben Familie und desselben Präfixes werden dabei hierarchisch relevant betrachtet. Es zeigt sich jedoch keine Verbesserung gegenüber dem exakten Targettraining. Der Recall@1 sinkt von 32,41 % auf 31,62 % und die MRR von 44,18 % auf 43,28 %. Die dadurch entstandene reichhaltigere Zieldefinition hat somit keinen positiven Einfluss auf die Modellleistung, (Alstetter 2026).

9.1.6 Profilbreite

In den Konfigurationen A06 und A07 werden unterschiedliche Profilbreiten untersucht. Bei A06 beträgt diese zwölf, bei A07 acht. Die Erhöhung der Profilbreite führt zu einer Verschlechterung des Recall@1 von 32,41 % auf 31,62 % und der MRR von 44,18 % auf 43,74 %. Eine Verringerung der Profilbreite führt zu keiner nennenswerten Veränderung der Modellleistung. Die Ergebnisse zeigen, dass eine verringerte Profilbreite die Modellleistung nicht signifikant verbessert, (Alstetter 2026).

9.1.7 Regularisierung

Die deutlichsten Verbesserungen werden durch die Veränderung der Regularisierung erzielt. A08 verwendet eine mittlere L1- und L2-Regularisierung ($\alpha = 0{,}1$, $\lambda = 0{,}5$). Dadurch wird mit einem Recall@1 von 0,3478 der höchste Top-1-Wert aller untersuchten Konfigurationen erreicht. Auch der höchste MRR-Wert von 0,4534 wird durch A08 erreicht. Mit A09 wird die Regularisierung nochmals deutlich erhöht ($\alpha = 0{,}5$, $\lambda = 1{,}0$). Zusätzlich wird ein positiver Wert für `min_split_gain` gesetzt, um die Stabilität der Baumaufteilung zu erhöhen. Diese Konfiguration erreicht den höchsten Recall@10 von 0,6917 und die zweithöchste MRR von 0,4505.

Die L1/L2-Regularisierung wirkt sich bei LightGBM direkt auf die Blattgewichte aus. Sie dämpft diese bei geringer Aussagekraft auf null und unterdrückt dadurch das Rauschen der 2.381 Merkmale bei nur 984 Trainingsbeispielen, ohne das schwache Unterscheidungssignal zwischen Makrofamilien-Varianten zu verlieren. Bei A09 wird durch die stärkere Dämpfung auch dieses Signal mit reduziert, was die Top-1-Präzision verringert, dabei aber die Gesamtrangfolge robuster macht, (Alstetter 2026).

9.1.8 Top-Konfiguration

Die beste Konfiguration der Studie ist A08, da sie sowohl den höchsten Recall@1 als auch den höchsten MRR erreicht(Alstetter 2026).

Die wichtigsten Kennzahlen dieser Variante sind in Tabelle 9.5 dargestellt.

Tabelle 9.5: Ergebnisse der besten Konfiguration A08

Kennzahl	**Wert**
Recall@1	0,3478
Family Recall@1	0,8735
Recall@10	0,6798
Family Recall@10	0,9368
MRR	0,4534
Mittlerer Zielrang	23,96
90 %-Perzentil	61,6
Schlechtester Rang	360

Die Fehleranalyse dieser Konfiguration zeigt, dass 7,9 % aller Anfragen einer fachlich falschen Makrofamilie zugeordnet wurden. 52,6 % der Fehlklassifikationen erfolgen innerhalb der korrekten Makrofamilie. Die größte Herausforderung liegt damit weiterhin in der Variantenauflösung technisch ähnlicher Makrofamilien, (Alstetter 2026).

10 Diskussion der Ergebnisse

10.1 Interpretation der Ergebnisse

Nachdem die Modellauswahl abgeschlossen wurde, erfolgt die finale Evaluation des ausgewählten Modells auf dem unberührten Testsplit.

Tabelle 10.1 zeigt die erzielten Ergebnisse des Referenzlaufs A08 auf dem Testsplit.

Tabelle 10.1: Ergebnisse des Referenzlaufs A08 auf dem Testsplit

Metrik	Wert
Target Recall@1	0,3478
Target Recall@3	0,4783
Target Recall@5	0,5494
Target Recall@10	0,6798
Family Recall@1	0,8735
Family Recall@10	0,9368
MRR	0,4534
Hierarchical nDCG@10	0,7449

Beim Testsplit zeigt sich ein ähnliches Bild wie beim Validierungssplit. Das Modell sagt bei 34,78 % der Anfragen das korrekte Ziel-Makro (EPLAN GmbH & Co. KG 2025) auf Platz 1 voraus, während die korrekte Makrofamilie in 87,35 % der Anfragen auf die erste Position gesetzt wird.

Wie beim Validierungssplit ist auch hier der deutliche Abstand zwischen Family Recall@10 (93,68 %) und Target Recall@10 (67,98 %) zu erkennen.

Der erreichte MRR-Wert von 45,34 % sowie die hierarchische nDCG@10 von 74,49 % bestätigen zusätzlich die Erkenntnisse aus dem Validierungssplit, (Alstetter 2026).

10.2 Ursachenanalyse der Rankingfehler

Um die Modellleistung detailliert erfassen zu können, werden die Top-1-Ergebnisse des Testsplits analysiert. Dabei werden die fachlichen Beziehungen zum tatsächlichen Ziel-Makro untersucht.

Tabelle 10.2 zeigt die Verteilung der einzelnen Fehlerkategorien.

Tabelle 10.2: Top-1-Analyse der Vorhersagen auf dem Testdatensatz

Kategorie	**Anzahl**	**Anteil**
Exaktes Ziel-Makro (EPLAN GmbH & Co. KG 2025)	88	34,78 %
Gleiche Familie, falsches Target	133	52,57 %
Gleiches Familienpräfix	12	4,74 %
Andere Familie	20	7,91 %

Die Fehleranalyse bestätigt die Beobachtungen aus den Metriken. Von insgesamt 253 Testanfragen werden 133 Fälle einer gleichen Familie, aber einer falschen Variante zugeordnet, was 52,57 % entspricht. In 7,91 % der Fälle wird eine andere Familie vorhergesagt. Das zeigt, dass das Modell die grundlegenden technischen Zusammenhänge zwischen Artikeln und Makrofamilien zuverlässig erlernt hat. Die Analyse zeigt, dass die zentrale Herausforderung die Identifikation des korrekten Ziel-Makros (EPLAN GmbH & Co. KG 2025) innerhalb der richtigen Makrofamilie ist. Dafür muss die Variantenauflösung innerhalb der Makrofamilie weiter verbessert werden. Die große Differenz zwischen dem Target Recall@1 und dem Family Recall@1 unterstreicht diese Herausforderung zusätzlich. Dies wird im Kapitel 11 genauer analysiert, (Alstetter 2026).

11 Grenzen und Ausblick

11.1 Grenzen des aktuellen Systems

Die Ergebnisse der Evaluierung zeigen, dass die größte Einschränkung des aktuellen Systems in der Variantenauflösung technisch ähnlicher Makrofamilien liegt. Dies lässt sich auf die Struktur sowie die Eigenschaften der verfügbaren Datenbasis zurückführen.(Alstetter 2026)

Tabelle 11.1 fasst die zentralen Grenzen des Systems und die daraus abgeleiteten Ansatzpunkte zusammen.

Tabelle 11.1: Übersicht der zentralen Grenzen und Ansatzpunkte

Grenze	**Beobachteter Befund**	**Ansatzpunkt**
Geringer Support	302 Kandidatenprofile basieren nur auf einem Trainingsartikel	Erweiterung der Datenbasis für seltene Ziel-Makros
Variantenauflösung	Family Recall@1 liegt deutlich über Target Recall@1	Spezialisierte Modellierung innerhalb der Makrofamilien
Support-Bias	Häufige Ziel-Makros besitzen stabilere Kandidatenprofile	Ausgleich seltener Profile durch zusätzliche Beispiele und Profilpflege

11.1.1 Geringer Support von Ziel-Makros

Eine zentrale Einschränkung ergibt sich aus der Support-Verteilung der Kandidatenprofile. Von insgesamt 375 vorhandenen Kandidatenprofilen basieren 302 Profile auf nur einem Trainingsartikel. Der Median-Support liegt bei 1 und der Mittelwert der Artikel pro Ziel-Makro (EPLAN GmbH & Co. KG 2025) beträgt 2,62.

Ein einzelner Artikel ist dabei nicht in der Lage, einen ausreichenden Support für die technische Vielfalt innerhalb einer Makrofamilie zu bieten. Die daraus entstandenen Kandidatenprofile sind somit von nur einem Artikel abhängig.

11.1.2 Variantenauflösung innerhalb einer Makrofamilie

Die Resultate der Evaluierung zeigen, dass das Modell die korrekte Makrofamilie deutlich zuverlässiger erkennt als das exakte Ziel-Makro (EPLAN GmbH & Co. KG 2025). A08 als beste Konfiguration erzielt dabei einen Family Recall@1 von 87,35 %, wohingegen der Target Recall@1 lediglich 34,78 % beträgt.

Zudem zeigt die Fehleranalyse, dass in 133 Testfällen die korrekte Makrofamilie erkannt wird, jedoch nicht das exakte Ziel-Makro (EPLAN GmbH & Co. KG 2025). Die größte Herausforderung liegt somit in der fehlerfreien Unterscheidung zwischen sehr ähnlichen Varianten innerhalb derselben Makrofamilie (Alstetter 2026).

11.1.3 Support-Bias häufiger Ziel-Makros

Der Support-Bias bezeichnet einen Rankingvorteil von Ziel-Makros mit hohem Kandidaten-Support. Da die Kandidatenprofile dieser Ziel-Makros auf mehreren Trainingsartikeln beruhen, können dadurch breitere technische Merkmale abgedeckt werden und dadurch häufiger durch das Modell gefunden werden.

Die durchgeführten Fehleranalysen zeigen einen deutlichen Support-Bias. In 55,6 % der Fälle besitzt das fälschlich ausgewählte Ziel-Makro (EPLAN GmbH & Co. KG 2025) einen höheren Support als das korrekte Ziel-Makro (EPLAN GmbH & Co. KG 2025). Im Durchschnitt verfügt das falsch gewählte Ziel-Makro dabei über etwa 10,6 Artikel mehr als das korrekte Ziel-Makro (Alstetter 2026).

Häufige Ziel-Makros (EPLAN GmbH & Co. KG 2025) erzeugen dadurch stabilere Kandidaten-profile und bekommen somit einen strukturellen Vorteil gegenüber selteneren Varianten.

11.2 Ausblick

Die Ergebnisse dieser Arbeit zeigen mehrere Ansatzpunkte für künftige Weiterentwicklungen auf.

Die größte erwartbare Leistungssteigerung liegt in der Erweiterung der vorhandenen Datenbasis. Besonders für seltene Ziel-Makros (EPLAN GmbH & Co. KG 2025) sollte eine größere Basis an Trainingsbeispielen geschaffen werden. Dadurch lässt sich die heutige Dominanz von Singleton-Profilen reduzieren und der Raum der Kandidaten-Profile würde eine stabilere Struktur aufweisen. Dies würde dazu führen, dass die breitere Datenbasis dem Modell mehr Informationen für eine bessere Generalisierung zur Verfügung stellt.

Ein weiterer Optimierungspunkt liegt in der Verbesserung der Variantenauflösung innerhalb einer Makrofamilie. Hierfür sind detailliertere Artikelmerkmale erforderlich, die es ermöglichen,

die einzelnen Artikel präziser voneinander abzugrenzen. Des Weiteren wäre der Ansatz einer mehrstufigen Architektur mit einem globalen Modell zur Familienerkennung und einem weiteren separaten Modell zur Variantenauflösung innerhalb der Makrofamilie denkbar. Dadurch könnten dann auch eventuelle Fehlerfälle zwischen der Familienerkennung und der Variantenauflösung systematischer analysiert und kompensiert werden.

Weiterführend stellt die Extraktion von Merkmalen auch einen interessanten Ansatz dar. Seltene oder redundant genutzte Merkmale könnten gezielt identifiziert und reduziert werden. Dadurch würde die Modellkomplexität verringert und die Generalisierungsfähigkeit des Modells verbessert werden.

Eine weitergehende Idee für die Zukunft wäre die Integration von zusätzlichem Domänenwissen von Experten oder externen Wissensquellen, welche zusätzliche technische Merkmale, aber auch semantische Zusammenhänge zwischen den Artikeln bereitstellen könnten. Diese Kombination aus Daten und Metadaten könnte einen vielversprechenden Ansatz für die Modellierung von spezifischeren Variantenunterscheidungen darstellen.

12 Zusammenfassung und Fazit

Das Ziel dieser Arbeit war die Entwicklung sowie Evaluierung eines datengetriebenen Systems zur automatisierten Zuordnung von EPLAN-Artikeln (EPLAN GmbH & Co. KG 2026) zu EPLAN-Makros (EPLAN GmbH & Co. KG 2025). Die zugrunde liegende Problemstellung wurde als eine Learning-to-Rank-Aufgabe formuliert, bei der das Modell lernen soll, die relevantesten Ziel-Makros (EPLAN GmbH & Co. KG 2025) für einen gegebenen EPLAN-Artikel (EPLAN GmbH & Co. KG 2026) zu priorisieren. Dabei soll eine Rangliste möglicher Ziel-Makros erzeugt werden.

Hierzu wurde eine End-to-End-Pipeline entwickelt, welche die Datenaufbereitung, die Erzeugung von Kandidatenprofilen, die Berechnung strukturierter Anfrage-Kandidat-Merkmale, das Training eines LightGBM LambdaRank-Modells sowie die automatische Evaluation umfasst. Die Evaluierung der entwickelten Lösung erfolgte anhand eines realen Datensatzes von EPLAN-Artikeln (EPLAN GmbH & Co. KG 2026) und den zugehörigen EPLAN-Makros (EPLAN GmbH & Co. KG 2025).

Die Datenbasis umfasst 1.559 EPLAN-Artikel (EPLAN GmbH & Co. KG 2026), 375 Ziel-Makros (EPLAN GmbH & Co. KG 2025) und 55 Makrofamilien. Aus diesen Daten wurden anschließend 375 Kandidatenprofile erzeugt und durch 2.381 technische, lexikalische und strukturierte Merkmale beschrieben. Die Trainingsmatrix umfasst 369.000 Anfrage-Kandidat-Paare und bildet damit die Grundlage für das Training des LightGBM-LambdaRank-Modells.

Es wurden neun unterschiedliche Modellvarianten im Rahmen einer systematischen Konfigurationsstudie untersucht. Dabei wurden verschiedene Familiengewichtungen, Labeldefinitionen, Profilbreiten und Regularisierungsstufen bewertet. Die beste Leistung wurde dabei durch die Konfiguration A08 erzielt, welche einen Target Recall@1 von 34,78 %, einen Family Recall@1 von 87,35 %, einen Target Recall@10 von 67,98 % sowie einen Mean Reciprocal Rank von 45,34 % aufweist.

Die Ergebnisse zeigen, dass das entwickelte System in der Lage ist, die korrekte Makrofamilie in 8 bis 9 von 10 Fällen korrekt zu erkennen. Die Analyse der vorhandenen Fehler zeigte gleichzeitig, dass die größere Herausforderung in der präzisen Erkennung der Variantenunterscheidung innerhalb derselben Makrofamilie liegt. Mehr als die Hälfte aller Fehlklassifikationen entsteht dabei innerhalb der korrekten Familie, während fachlich unverbundene Familien nur einen geringen Anteil der Fehler verursachen.

Die Häufigkeiten der Ziel-Makros folgen dabei einer sogenannten Long-Tail-Verteilung. Darunter wird eine stark ungleichmäßige Verteilung verstanden. Dabei kommen wenige Ziel-Makros

sehr häufig vor, während viele weitere Ziel-Makros nur selten oder einmalig vorkommen. Die durchgeführten Analysen zeigen außerdem, dass sich die Leistungsgrenzen des entwickelten Systems primär durch diese Long-Tail-Verteilung mit 213 Singleton-Targets und 279 Ziel-Makros (EPLAN GmbH & Co. KG 2025) mit weniger als drei Trainingsbeispielen ergeben. Diese erschweren den Aufbau robuster Kandidatenprofile. Die zusätzliche Leave-One-Out-Analyse zeigt darüber hinaus, dass die ursprünglich vermutete Generalisierungslücke durch die Konstruktion der Kandidatenprofile verursacht wurde. Demgegenüber verhält sich die Generalisierung zwischen Validierungs- und Testdaten stabil und liefert keine Hinweise auf ein kritisches Overfitting des verwendeten LightGBM-LambdaRank-Modells.

Zusammenfassend konnte nachgewiesen werden, dass der entwickelte Learning-to-Rank-Ansatz eine geeignete Grundlage für die automatisierte Zuordnung von EPLAN-Artikeln (EPLAN GmbH & Co. KG 2026) zu EPLAN-Makros (EPLAN GmbH & Co. KG 2025) darstellt.

Die künftigen Leistungssteigerungen sind vielmehr durch eine verbesserte Datenbasis als durch zusätzliche Hyperparameter-Optimierungen zu erwarten. Zusätzliche Variantenbeispiele sowie spezialisierte Verfahren zur Variantenauflösung innerhalb der Makrofamilien können die zukünftige Leistungsfähigkeit des Systems weiter verbessern.

Somit kann die Forschungsfrage trotz der identifizierten Limitationen und Herausforderungen des Systems als positiv beantwortet werden, da nachgewiesen werden konnte, dass der limitierende Faktor nicht die Modellarchitektur, sondern die Datenbasis darstellt.

Abbildungsverzeichnis

4.1 Verzahnung von DSRM und ADR. Die DSRM bildet den übergeordneten Forschungsrahmen, während ADR als iterativer Entwicklungszyklus innerhalb der Phase „Design & Entwicklung des Artefakts“ eingesetzt wird. 8

Tabellenverzeichnis

8.1 Dimensionen der erzeugten Rankingmatrizen 13
8.2 Variierte Hyperparameter der Grid-Search (Pipeline step 931 2026) 15

9.1 Untersuchte Konfigurationsparameter 16
9.2 Referenzkonfiguration A01 17
9.3 Abweichungen der Konfigurationen gegenüber der Referenz A01 18
9.4 Ergebnisse der A01–A09-Konfigurationen auf dem Testdatensatz 18
9.5 Ergebnisse der besten Konfiguration A08 20

10.1 Ergebnisse des Referenzlaufs A08 auf dem Testsplit 21
10.2 Top-1-Analyse der Vorhersagen auf dem Testdatensatz 22

11.1 Übersicht der zentralen Grenzen und Ansatzpunkte 23

Literaturverzeichnis

Alstetter, Andreas (2026). *LTR Configuration Test Series*. Eigene Untersuchung. Interne Konfigurationsanalyse der LTR-Modelle (siehe S. 16–24).

EPLAN GmbH & Co. KG (2025). *Makros*. EPLAN-Plattform Infoportal (siehe S. 19, 21–24, 26, 27).

EPLAN GmbH & Co. KG (2026). *Artikelverwaltung*. EPLAN-Hilfe (siehe S. 26, 27).

Järvelin, Kalervo und Jaana Kekäläinen (2002). „Cumulated Gain-Based Evaluation of IR Techniques“. In: *ACM TOIS*. Bd. 20. 4, S. 422–446. DOI: 10.1145/582415.582418 (siehe S. 4, 5).

Liu, Tie-Yan (2009). „Learning to Rank for Information Retrieval“. In: *Foundations and Trends in Information Retrieval* 3.3, S. 225–331. DOI: 10.1561/1500000016 (siehe S. 2).

Manning, Christopher D., Prabhakar Raghavan und Hinrich Schütze (2008). *Introduction to Information Retrieval*. Cambridge University Press (siehe S. 3).

Microsoft Corporation (2026). *LightGBM*. Microsoft Corporation (siehe S. 13).

Peffers, Ken, Tuure Tuunanen, Marcus A. Rothenberger und Samir Chatterjee (2007). „A Design Science Research Methodology for Information Systems Research“. In: *Journal of Management Information Systems* 24.3, S. 45–77. DOI: 10.2753/MIS0742-1222240302 (siehe S. 7).

Pipeline step 931 (2026). *Pipeline step 931: Learning-to-Rank macro ranker training*. Interner Projektquellcode (siehe S. 14, 15).

Sein, Maung K., Ola Henfridsson, Sandeep Purao, Matti Rossi und Rikard Lindgren (2011). „Action Design Research“. In: *MIS Quarterly* 35.1, S. 37–56. DOI: 10.2307/23043488 (siehe S. 7).

Voorhees, Ellen M. und Dawn M. Tice (1999). „The TREC-8 Question Answering Track Evaluation“. In: *Proceedings of the Eighth Text REtrieval Conference (TREC-8)*. DOI: 10.6028/NIST.SP.500-246.qa-overview (siehe S. 3).

Zhu, Yutao, Huaying Yuan, Shuting Wang, Jiongnan Liu, Wenhan Liu, Chenlong Deng, Haonan Chen, Zheng Liu, Zhicheng Dou und Ji-Rong Wen (2024). „Large Language Models for Information Retrieval: A Survey“. In: *arXiv preprint arXiv:2308.07107* (siehe S. 3).